GitHub Lab Manual – Fork → Clone → Commit → Push (Beginner-friendly)

This guide is for **first-time GitHub users**. It shows students exactly what to click/type.

---

0) What you'll do in this lab

Flow: Teacher posts a repo for each task → Student **Forks** it → **Clones** fork to laptop → works locally → **Commits** changes → **Pushes** to *their* fork (and shows your teacher during lab).

Key words:

- **Fork**: Your own copy of the teacher's repo on your GitHub account.
- **Clone**: Download the repo to your computer.
- **Commit**: Save a snapshot of changes with a message.
- **Push**: Send commits from your laptop to GitHub.
- **Upstream**: The teacher's original repo.

---

1) Student Quick Start (one-page)

1. Create/Log in to **GitHub**.
2. Open the task repo link from your instructor → Click **Fork** → leave the default options → **Create fork**.
3. On your fork, click **Code** → **HTTPS** → copy the URL.
4. On your computer, open **VS Code** → **View → Terminal**.
5. Run:

```
git clone https://github.com/<your-username>/<task-repo>.git
cd <task-repo>
```

1. [**First time using GitHub only it may ask**] Tell Git who you are :

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

1. Link to the teacher's repo as `upstream` (so you can pull updates):

```
git remote add upstream https://github.com/<teacher-org-or-user>/<task-repo>.git
git remote -v
```

1. Create a working branch:

```
git checkout -b lab1-solution
```

1. Do the task. Save files.
2. Stage + commit + push:

```
git add .
git commit -m "feat: complete part A – implemented X"
git push -u origin lab1-solution
```

1. Confirm on GitHub: your fork now shows your branch & commits.

   Submission for this lab: Show your fork/branch to your instructor in lab and run the app.

---

2) [Skip if done no.1] Step-by-step with VS Code UI (no terminal required)

1. **Fork** on GitHub (same as above).
2. In VS Code: **Source Control** (icon on left) → **Clone Repository** → paste your fork URL → choose folder.
3. Open folder → make changes.
4. **Source Control** → type a commit message → **Commit**.
5. **Sync Changes** (or **Push**) when prompted.
6. To pull teacher updates later: **Source Control** → **…** menu → **Remote** → add `upstream` URL (teacher repo) → **Pull from… upstream** → merge.

---

3) [Skip in labs]Optional: Create a Pull Request (PR)

You may need this later in your project with your team mates

1. Push your branch to your fork (done above).
2. On GitHub, click **Compare & pull request**.
3. Base repo = teacher's repo, base branch = `main`. Head repo = your fork, head branch = `lab1-solution`.
4. Title: `Lab 1 – <Your Name>`. Description: what you did, how to test, screenshots.
5. Click **Create pull request**.

---

4) Naming rules & commit style (keep it clean)

- Repo names from teacher: `lab01-perks`, `lab02-api`, etc.
- Student branches: `lab1-solution`, `fix/bug-123`, `feat/add-search`.
- Commits: use short prefixes like `feat:`, `fix:`, `docs:`, `test:`, `refactor:`.

Examples:

```
feat: implement discount filter
fix: handle 404 on GET /perks/:id
refactor: extract db connection into module
```

---

5) What you'll show the instructor (submission checklist)

- ✅ Your **fork link** on GitHub.
- ✅ Your **working branch** visible (`lab1-solution`).
- ✅ App runs locally (show terminal output or UI).
- ✅ Clean commit history with meaningful messages.

---

6) Keeping your fork up-to-date (details)

If teacher updates the starter code after you forked:

```
git fetch upstream
# update your local main with teacher's main
git checkout main
git merge upstream/main
# push your updated main to your fork
git push origin main
# bring updates into your work branch
git checkout lab1-solution
git merge main
```
